The Forte Programming Language

(define-syntax (uncle-sams-holy-trinity stx)
#"(Aws Luke Xiao))

Introduction to Forte

Forte is an esoteric programming language intended to add executability to a musical score.

A Forte program consists of a staff that contains defined chords. The abstract syntax consists of:

```
(staff <Chord> …)

(chord <Note> …)
```

Chords are where most of Forte's functionality resides.

Forte interprets a series of musical chords as executable instructions, and individual notes are variables that can hold values. e.g. `(chord A1 A2 A3)` corresponds to addition.

The Forte instruction set allows for arithmetic, conditionals, functions, while loops, and basic output.

There are 88 possible variables that can be written to. One for each of the 88 keys on a piano.

Notes (and chords) are written using Scientific Pitch Notation.

An example program can be written as:

```
(staff
 (chord A1 A2 A3)    // instruction
 (chord D1 D2)       // args
 (chord D1))         // output
```

Macro Expansion

The macro system expands the input music score into a nested list, Which is the input of the `interp` function.

```
(staff
  (chord D2 F2 G2)
  (chord B1 B2 B3))
```

```
(interp
  (list (list 'D2 'F2 'G2)    ; cmd: RESET
        (list 'B1 'B2 'B3))) ; args: B1 B2 B3
```

A staff is a list of chords. Each chord is a list of notes. Each note is a symbol.

```
(define-syntax (my-module-begin stx)
  (syntax-parse stx
    #:datum-literals (staff)
    [(_)
     #'(#%module-begin)]
    [(_ expr ...)
     #'(#%module-begin (interp expr) ...)]))

(define-syntax (staff chords)
  (syntax-parse chords
    #:datum-literals (chord)
    [(_ (chord notes ...) ...)
     #'(list (list 'notes ...) ...)]))
```

The Interpreter

The challenge: no explicit boundary between commands.

- Command names and args are both chords.
- No syntactic separator between commands.
- Loops and functions must be structured – nested loops and functions should be possible.

```
(interp
  (list (list 'A5 'A6 'A7)    ; cmd: RESET
        (list 'B1 'B2 'B3))) ; arg: B1 B2 B3
```

Our solution: treat the interpreter as a state machine.

- State determines if a chord is a command name or command argument.
- For code organization, the main interpreter passes to specialized helper functions.

```
(define (interp insts)
  (match insts
    ['() (void)]
    [(cons inst rest)
     (match state
       ['NONE
        (set-state inst)
        (interp rest)]
       [(or 'RESULT 'CMD_ESTABLISH 'CMD_RESET 'CMD_ASSIGN)
        (interp-set insts)]
       [(or 'CMD_ADD 'CMD_SUB 'CMD_MUL 'CMD_DIV 'CMD_BITWISE_AND)
        (interp-arith insts)]
       [(or 'CMD_START_FUNC_DEF 'FUNCTION_CALL)
        (interp-func insts 0)]
       [(or 'CMD_PRINT_INT 'CMD_PRINT_CHAR)
        (interp-print insts)]
       [(or 'CMD_IF)
        (interp-cond insts (Cond empty empty empty) 0)]
       [(or 'CMD_WHILE)
        (interp-while insts (While empty empty) 0)]
       [else (error state)])]))
```

Sample Program I: Cond

1. Confront IF, enter *interp-cond()*
2. Collect all commands until THEN as Cond-cond.
3. Collect all commands until ELSE as Cond-then.
   An *index* is used to record nest level.
4. Collect all commands until END_IF as Cond-else.
5. Interp the condition to determine which branch to enter.
6. Interp branch and then continue on remaining chords.

```
(staff
 (chord G2 A3 B3)              ; IF
 (chord C2 F2) (chord D1 D3) ;   EQUAL D1 = D3
 (chord G2 B3 C3)              ; THEN
 (chord D2 E2 F2) (chord A1) ;   ESTABLISH (SET ONE) A1
 (chord G2 A3 B3)              ;   IF
 (chord C2 F2) (chord D1 D3) ;     EQUAL D1 = D3
 (chord G2 B3 C3)              ;   THEN
 (chord E2 F2 G2) (chord A1) ;     PRINT_INT A1
 (chord G2 C3 D3)              ;   ELSE
 (chord E2 F2 G2) (chord D3) ;     PRINT_INT D3
 (chord G2 D3 E3)              ;   END_IF
 (chord G2 C3 D3)              ; ELSE
 (chord E2 F2 G2) (chord A1) ;   PRINT_INT A1
 (chord G2 D3 E3))             ; END_IF
```

Sample Program II: Loop

1. Confront WHILE, enter *interp-cond()*.

2. Collect While-cond and While-body
   An *index* is used to record nest level.

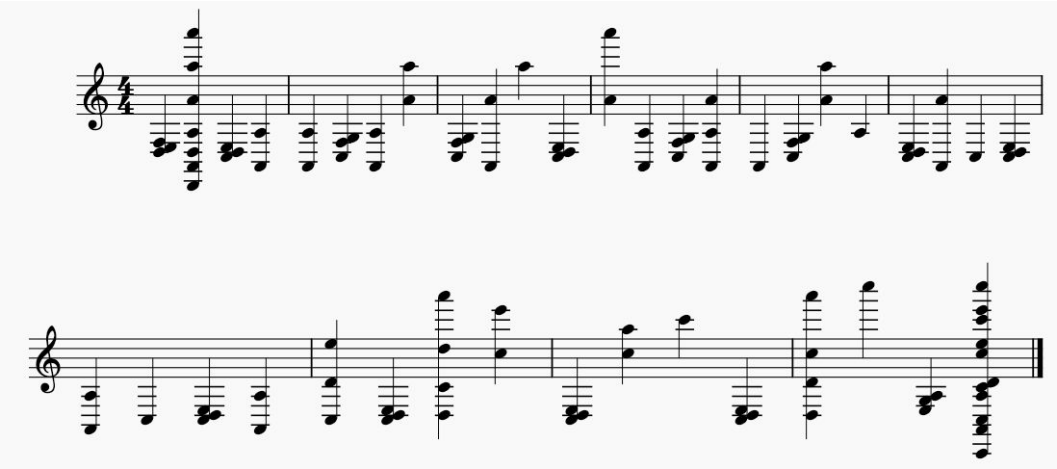
3. Interp While-cond and loop While-body while it holds.

4. Interpret remaining chords.

```
(staff
 (chord D2 E2 F2) (chord D1 D2 D5)              ; SET_ONE D1, D2, D5
 (chord C2 D2 E2) (chord D1 D2) (chord D1 D2)   ; D1 = D2 = ADD D1 D2
 (chord C2 E2 F2) (chord D1 D2)                 ; tmp = MUL D1 D2
 (chord D1 D2)                                  ; D1 = D2 = tmp
 ; D1 = 4, D2 = 4, D3 = 0, D4 = 0, D5 = 1
 (chord G2 A3 B3)                             ; WHILE
 (chord D2 F2) (chord D3 D1)                  ;   D3 < D1
 (chord C3 D3 E3)                             ; WHILE_BODY
 (chord G2 A3 B3)                             ;   WHILE
 (chord D2 F2) (chord D4 D1)                  ;     D4 < D1
 (chord C3 D3 E3)                             ;   WHILE_BODY
 (chord E2 F2 G2) (chord D4)                  ;     PRINT_INT D4
 (chord C2 D2 E2) (chord D4 D5) (chord D4)    ;     D4 = D4 + D5
 (chord F3 A3 C3)                             ;   END_WHILE
 (chord C2 D2 E2) (chord D3 D5) (chord D3)    ;   D3 = D3 + D5
 (chord D2 G2 C3) (chord D4)                  ;   SET_ZERO D4
 (chord F3 A3 C3)                             ; END_WHILE
 )
```

Live Demo

A survey of what is possible with *Forte*!



“Hello World”